

Probing In probing experiments (Shi et al., 2016), we fix the target LM, then train a smaller “probe” model (e.g. a linear classifier) to map LM hidden representations h to quantities z hypothesized to be encoded by the LM (Figure 1C). Our experiments specifically evaluate whether (1) the state s_t , and (2) the final state parity is linearly encoded in intermediate-layer representations. For each layer l , we train (1) a state probe to predict $p(s_t | h_{t,l})$ and (2) a parity probe to predict $p(\epsilon(s_t) | h_{t,l})$. Given a trained LM, we collect representations on one set of input sequences to train the probe, then evaluate probe accuracy on a held-out set.

Activation Patching Probing experiments reveal what information is *present* in an LM’s representations, but not that this information is *used* by the LM during prediction. Activation patching is a method for determining which representations play a *causal* role in prediction. Portions of the LM’s internal representations are overwritten (“patched”) with representations derived from alternative inputs; if predictions change, we may conclude that the overwritten representations was used for prediction (Meng et al., 2022; Zhang & Nanda, 2024; Heimersheim & Nanda, 2024).

Let $p(y | x; h \leftarrow h')$ denote the probability that an LM assigns to the output y given an input x , but with the representation h replaced by some other representation h' . In a typical experiment, we first construct a “clean” input x that we wish to analyze, and a “corrupted” input x' that alters or removes information from x (e.g. by adding noise or changing its semantics). Next, we compute the most probable outputs from clean and corrupted inputs:

$$\hat{y} = \arg \max_y p(y | x)$$

$$\hat{y}' = \arg \max_y p(y | x')$$

We then re-run the LM on the corrupted input x' , but substitute a hidden representation from the clean input x , and measure how much prediction shifts toward the clean output $\hat{y}$ using the *normalized logit difference* (Wang et al., 2023):

$$\text{NLD} = \frac{\text{LD}(x'; h_{t,l} \leftarrow h_{t,l}^{\text{clean}}) - \text{LD}(x')}{\text{LD}(x) - \text{LD}(x')} \quad (2)$$

where

$$\text{LD}(\cdot) = \log p(\hat{y} | \cdot) - \log p(\hat{y}' | \cdot)$$

and the representation $h_{t,l}^{\text{clean}}$ is taken from the clean run of the model. A value of NLD close to 1 indicates that we have *restored* a part of the circuit that computes $\hat{y}$.

In this paper, we evaluate which representations are involved in prediction by presenting models with a clean sequence $[a_0, a_1, \dots, a_t]$ associated with a final state s_t . We then produce a corrupted sequence differing **only in the first token**, $[a'_0, a_1, \dots, a_t]$, associated with a final state

s'_t . We then identify the hidden states that, when patched in, cause the model to output s_t rather than s'_t with high probability. Our main experiments specifically perform *prefix patching*, where *all* hidden representations up to index t ($h_{1:t,l} \leftarrow h_{1:t,l}^{\text{clean}}$) are patched at a particular layer l (Figure 1B). Prefix patching allows us to localize how information gets progressively transferred to the final token as we move deeper into the network. A value close to 1 means that some part of the prefix representation was used for prediction; a value close to 0 means that *no* part was.

We also experiment with other types of localization techniques (including suffix and window patching), as well as zero-ablating certain activations in Section B.

3. What Algorithms Can Transformers Implement in Theory?

To use the methods described in §2.3 to interpret model behavior, we must first establish a *phenomenology* for LM state tracking—identifying candidate state tracking algorithms that might be implemented by the model, along with the empirical probing and activation patching results we would expect to find if these algorithms are implemented. Below, we describe a set of state tracking mechanisms suggested by the existing literature.

For each mechanism, we first present a sketch of an implementation, in the form of rules for computing the value stored in the hidden state for each layer and timestep. We then describe the “signature” of each algorithm—the result we would expect from the application of prefix patching and probing techniques described in the preceding section.

3.1. Sequential Algorithm

The sequential algorithm composes permutations one at a time from left to right (analogous to a mechanism some LMs use to solve multi-hop reasoning problems; Yang et al., 2024). Signatures of this algorithm would provide evidence that LMs implement step-by-step “simulation” in their hidden states to solve state tracking tasks. In this algorithm, each hidden state $h_{t,l}$ stores the associated action a_t until s_t can be computed, maintaining $h_{t,t} = s_t$. As shown in the first row of Figure 1D, this computation depends only on hidden states with $l \leq t$.

```

 $h_{t,0} = a_t \ \forall t$  // initialize actions
 $(h_{0,0} = s_t)$  // by definition; see §2.2
for  $t = 1..T, l = 1..L$  do
  if  $l < t$  then  $h_{t,l} = h_{t,l-1} = a_t$  // propagate actions
  if  $l = t$  then  $h_{t,l} = h_{t-1,l-1} h_{t,l-1}$ 
                                      $= s_{t-1} a_t = s_t$  // update states
  if  $l > t$  then  $h_{t,l} = h_{t,l-1} = s_t$  // propagate states
end for

```

Patching Signature Because of this dependency, any patching experiment that replaces only hidden states with $l > t$ will not affect the final model predictions, leading to the upper triangular patching signature shown in the first row of Figure 1B.

Probing Signature Because s_t can only be predicted at layer $l = t$, we expect a state probe to show a *linear* dependence on depth: for sequences of maximum length T , a probe at layer l will correctly label an l/T fraction of states. If these state representations linearly encode parity, then the accuracy of the parity probe will also increase linearly; otherwise, it will remain constant.¹

3.2. Parallel Algorithm

As noted in §2.2, the word problem on S_5 belongs to NC^1 (and thus requires a circuit depth that scales logarithmically with sequence length). The word problem on S_3 , however, belongs to TC^0 , the class of decision problems with constant-depth threshold circuits. See discussion in Section A and Merrill & Sabharwal (2023). A constant-depth circuit will give rise to a set of hidden-state dependencies like the second row of Figure 1D.

Patching Signature Let l_P denote the number of layers needed to implement the constant-depth circuit for this task. For patching interventions conducted at or earlier than layer l_P , we expect the model’s predictions to change; at deeper layers than l_P , interventions will have no effect at all, resulting in the L-shaped pattern shown in the second row of Figure 1B.

Probing Signature We expect the probe to obtain perfect accuracy within a constant number of layers. Because the algorithm described in Section A computes state parity as an intermediate quantity, the parity probe will also obtain perfect accuracy within a constant number of layers.

3.3. Associative Algorithm

In the associative algorithm (AA), Transformers compose permutations hierarchically: in each layer, adjacent sequences of permutations are grouped together and their product is computed. This is analogous to recursive scan in Liu et al. (2023) and flattened expression evaluation in Merrill et al. (2024). This algorithm takes advantage of the associative nature of the product of permutations, whereby $a_0 a_1 a_2 a_3 = (a_0 a_1)(a_2 a_3)$. It ensures that $h_{t,l} = a_{t-2^l+1} \cdots a_t$, and thus that $h_{t,\log(t+1)} = a_0 \cdots a_t$. Signatures of this algorithm would provide evidence that LMs perform state tracking not by encoding states, but rather by *mapping* between states, for prefixes of increasing length.

¹See Section C.3 for a representative model in which parity is not linearly decodable.

```

 $h_{t,0} = a_t \quad \forall t$  // initialize actions
for  $t = 0..T, l = 1..L$  do
  if  $l \leq \log(t+1)$  then
     $h_{t,l} = h_{t-2^{l-1},l-1} h_{t,l-1}$ 
     $= a_{t-2^{l-1}+1} \cdots a_t$  // compose actions
  else  $h_{t,l} = h_{t,l-1} = s_t$  // propagate actions
end for

```

(Defining $h_{t<0,l} = h_{0,l}$ for notational convenience.)

As seen in the third row of Figure 1D, the model’s prediction for s_t depends on the hidden representation $h_{t/2}$ in the layer before the final state is computed, the representation at $h_{t/4}$ in the layer before that, etc.

Patching Signature Consequently, for AA, the length of the prefix that must be modified to alter model behavior increases exponentially in depth, resulting in the signature in the third row of Figure 1B.

Probing Signature We similarly expect to see an exponentially increasing state probe accuracy (because s_t becomes predictable at layer $l = \log t$, a probe at layer l will correctly label a $2^l/T$ fraction of states). If state parity is encoded in state representations, then parity probe accuracy will also increase exponentially.

3.4. Parity-Associative Algorithm

In this algorithm (PAA), LMs compute the final state in two stages: first computing the parity of the state (which can be performed in a constant number of layers using a subroutine from the Parallel algorithm); then separately computing the remaining information needed to identify the final state (the “parity complement”) using a procedure analogous to AA. (Unlike the preceding algorithms, we are not aware of any previous proposals for solving permutation composition problems in this way; but as we will see, it is useful for understanding interactions between “heuristic” and “algorithmic” solutions in real LMs.)

We model implementation of PAA with hidden states comprising two “registers” ϵ and κ (i.e. $h_{t,l} = (\epsilon_{t,l}, \kappa_{t,l})$ which store the parity and complement respectively.

```

 $\kappa_{0,t} = a_t \quad \forall t$  // initialize actions
 $\epsilon_{0,t} = \text{par}(s_t) \quad \forall t$  // compute parities (App. A)
for  $t = 0..T, l = 1..L$  do
   $\epsilon_{t,l} = \epsilon_{t,l-1}$  // propagate parities
  if  $l \leq \log(t+1)$  then
     $\kappa_{t,l} = \text{comp}(\kappa_{t-2^{l-1},l-1} \kappa_{t,l-1})$  // compose
  else  $\kappa_{t,l} = \kappa_{t,l-1}$  // propagate complements
end for

```

In this algorithm, the hidden state at position i holds that

position’s state parity and parity complement (if computed at this point). Parity, like S_3 , may be computed with a constant number of layers. The algorithm sketch given above is deliberately vague about the implementation of the parity complement composition operation (comp). In practice, different representations of this complement appear to be learned across different runs; see Figure 10 for evidence that these representations are computed using a brittle (and perhaps heuristic- or memorization-based) mechanism.

Patching Signature If the corrupted input has a different parity from the clean input, then in layers deeper than those used to compute parity, it is necessary to restore the *entire* prefix to cause the LM to assign full probability to the clean prediction. On these inputs, prefix patching will show a signature similar to the parallel algorithm (see Figure 8B). However, if the corrupted input has the *same* parity as the clean input, the portion of the hidden state computed in parallel remains the same, while its complement is computed using the same mechanism as the associative algorithm (see Figure 8A). These inputs will thus exhibit an AA-like (exponentially-shaped) patching pattern. When averaged together, parity-matched and parity-mismatched patching will produce a pattern with two regions, one shaped like the associative algorithm (associated with a 50% restoration in accuracy) and one shaped like the parallel algorithm (associated with a 100% restoration in accuracy). Again, this may be most easily understood graphically (Figure 1).

Probing Signature We expect state probes to improve exponentially with depth, while parity probes converge to 100% at a constant depth.

4. What Mechanisms do Transformers Learn?

In this section, we compare these theoretical state tracking mechanisms to empirical properties of LMs trained for permutation tasks. It is important to emphasize that the various signatures described above provide necessary, but not sufficient, conditions for implementation of the associated algorithm; the exact *mechanism* that LMs use in practice is likely complex and dependent on other input features not captured by the algorithms described above.

Nevertheless, our experiments successfully rule out some possible state tracking mechanisms and identify algorithmic features likely to be shared between the idealized mechanisms above and the true behavior learned by transformers. Specifically, our experiments yield evidence consistent with the associative algorithm (AA) in some models and the parity-associative algorithm (PAA) in other models, across architectures, sizes, and initializations.

4.1. Experimental Setup

We generate 1 million unique length-100 sequences of permutations in both S_3 and S_5 . We split the data 90/10 for training/analysis, and fine-tune these models (using a cross-entropy loss) to predict the state corresponding to each *prefix* of each action sequence:

$$\mathcal{L} = - \sum_{t=0}^{99} \log p_{\text{LM}}(s_t \mid a_0 \dots a_t), \quad (3)$$

where $p_{\text{LM}}(s_n \mid a_0 \dots a_t)$ is the probability the language model places on state token s_n when conditioned on the length- n prefix of the document.

Except where noted, we begin with Pythia-160M models pre-trained on the Pile dataset (Biderman et al., 2023). Regardless of initialization scheme, we fine-tune models for 20 epochs on Equation (3) using the AdamW optimizer with learning rate 5e-5 and batch size 128. For larger models (above 700M parameters), we train using bfloat16.

4.2. Activation Patching

For both the S_3 and S_5 tasks, across training runs, we find that activation patching results exhibit two broad clusters of behavior. For some trained models, they match the activation patching signature associated with AA; in others, they match the signature of PAA—even when the only source of variability across training runs is the order in which data is presented. Results for prototypical AA- and PAA-type models, on both S_3 and S_5 , are shown in Figure 2. Additional patching results in Section B confirm that patching intermediate representations of PAA-type models (the light-

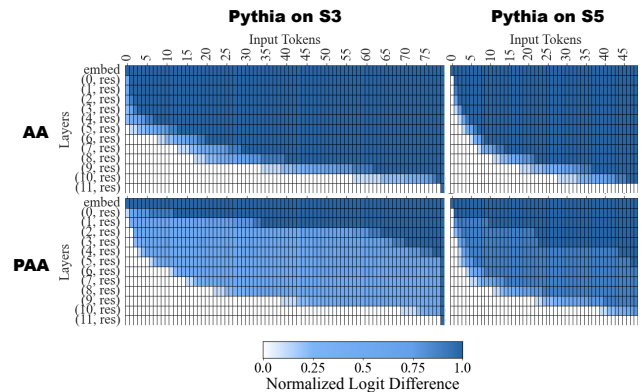


Figure 2. Activation patching on the residual stream for various Pythia models trained on S_3 and S_5 . Each cell at layer l and token t represents the probability of the correct final state when the *entire* prefix up to t at layer l is restored. We find signatures matching the AA and PAA algorithms from Figure 1, with both models ignoring exponentially longer prefixes as we traverse down the layers, and PAA models containing intermediate representations that encode some information about the final state, but not its parity.